
Making bitMoire

How I developed my own video synth with AI

Cursor to XYZ

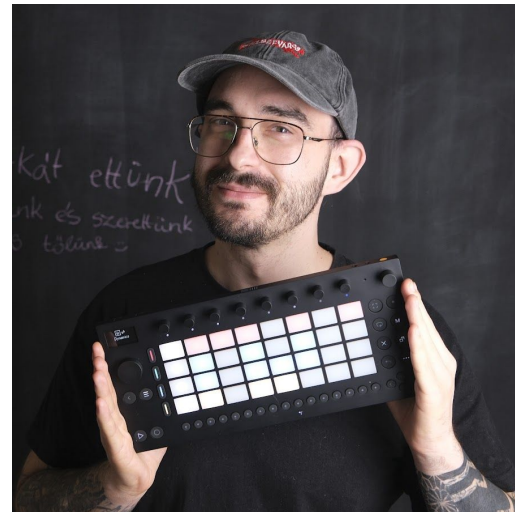
Making underground electronic music since 2009

Atmospheric, experimental, psychedelic

Improvisation with hardware, software, audio and video

Creating systems instead of songs or albums

Find me on the socials as @cursortoxyz



Why?

- Used to do game development as a kid using node based engines
- Working in tech for more than a decade, *but no hands-on coding anymore*
- AI agents made it possible to write code without learning the syntax
- I wanted to do A/V shows
- **...so why not build a video synth?**

Why not hardware?

Been there, done that

Too much space

Too expensive

Extra cables and components

Limited resolution (480p)

hydra + laptop + MIDI controller
can replace the entire stack



—

Development

From hydra-synth to gdrust

hydralyx

Generative visuals in the browser using Hydra Synth, Audio and MIDI input.

Use an Audio Input and MIDI device to control visuals in real-time.

Hydralyx

by @cursortoxSI

AUDIO



0%

Settings

Clear

Random Pattern

Previous Pattern

Next Pattern

Fullscreen

Render Resolution: 1280x720

Apply

Audio Input:

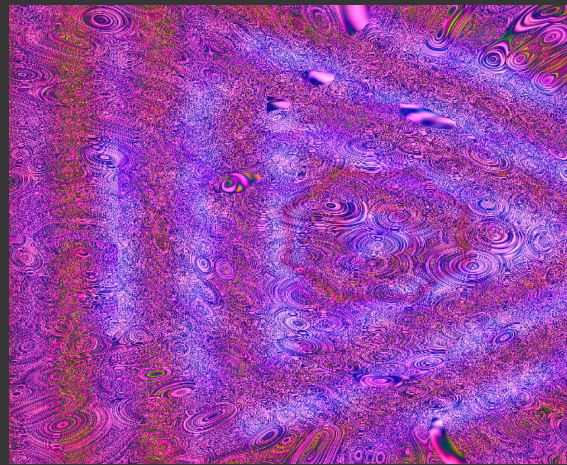
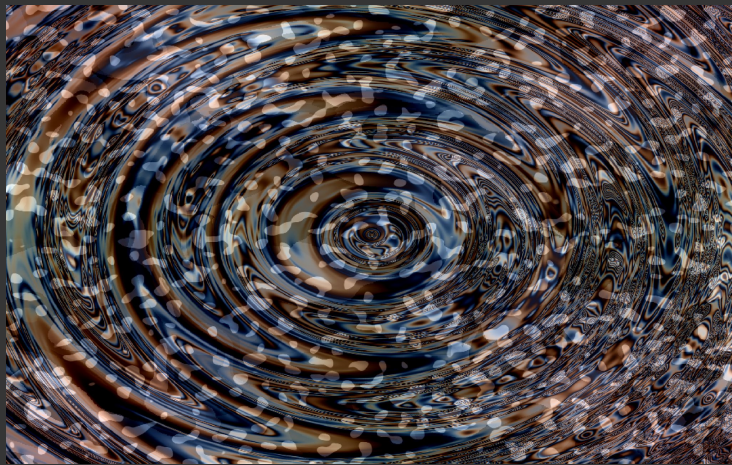
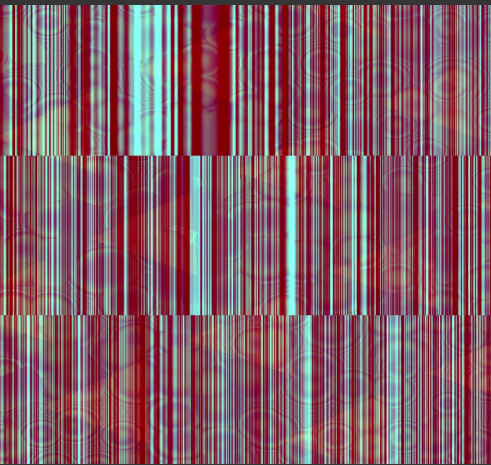
Click to select audio device...

Hydralyx

- Complete frontend rewrite
- Loads hydra patches at startup
- Patches include MIDI mapping
- It works, but inconvenient
- I don't like node + react

Hydralyx development

- Made with Qwen3-Coder-480B-A35B-Instruct
- IDE: Claude Code and Visual Studio Code
- Built feature-by-feature instead of trying to implement a long specification
- Lots of bugs, troubleshooting and debugging using chrome-web-tools
- Agents had a hard time testing the user interface



—

Why hydra rocks

Exposes easy to understand video synth abstractions like oscillators, mixers, etc

Removes the requirement to code shaders manually

Audio reactivity, MIDI extension exists

Lot of public patches

Great community

Multiple people had the similar idea on the interwebz

- **Hydralisk** - <https://github.com/delanni/hydralisk/tree/gh-pages>
- **HYDRCTRL** - <https://www.d17e.dev/projects/hydractrl/>

Bitgraves published a great post about how they use hydra for live performance

<https://reading.supply/@ben/creating-an-audiovisual-set-performing-with-hydra-while-also-performing-music-v5Enf1>



Why bitMoire?

Spent time prototyping pure CMOS logic synths in Bitwig

What would an early digital video computer look like?

Focus on aesthetics and limitations, but without *actual* limits

"Emulation" of a non-existing 80s flagship video synth

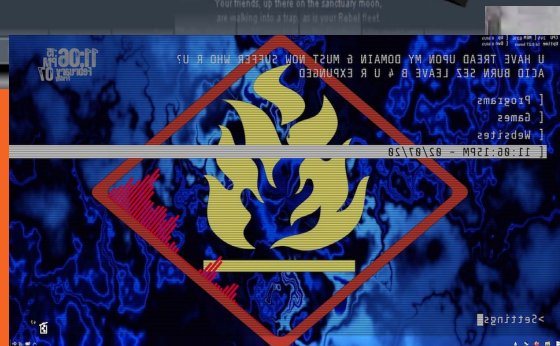
For example:

- Lo-fi digital oscillators
 - DX7 algorithm topology
 - CRT simulation
-

Inspiration

Generative Art

Demoscene/Cracktro



90s video games

pre-Y2K VFX

Glitch

Planning and Design



- Lots of brainstorming about potential approaches
- Wireframing UI on paper using grid systems
- Research era accurate technologies and solutions
- Think about what the software architecture should look like
- Just ideas - don't get too attached

Outcomes:

- I decided to emulate legacy systems in modern engine
- Full modular concept was not planned initially
- Created a mental model of how components should work
- Wireframe prototype that I could start implementing
- Decided to ditch the flagship 80s video synth idea



Philosophy

The following design principles were the key goals for bitMoire.

→ **Video synth for musicians**

Use synth-like signal path instead of node based workflow.

→ **Made for live play**

Inspired by software like Ableton Live, Bitwig Studio and Resolume.

→ **Modular at heart**

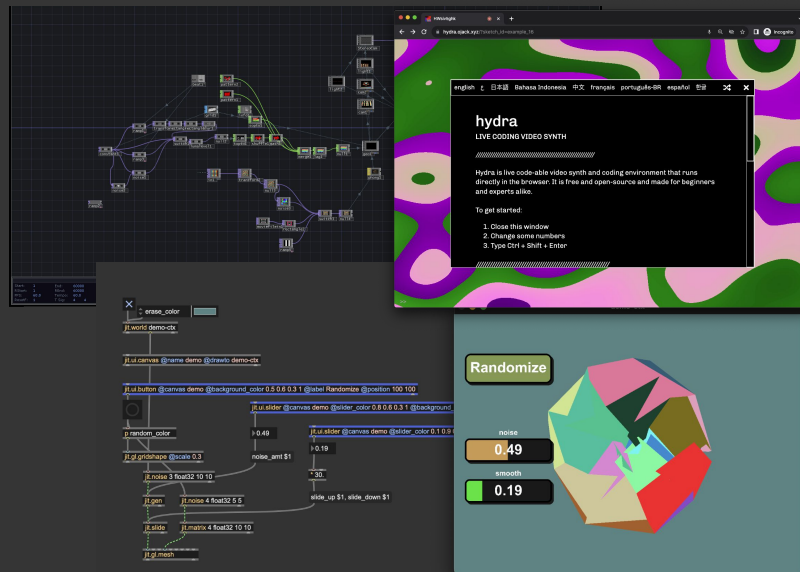
Opinionated signal sources, effects, modulators all playing together nicely.

Video synth for musicians

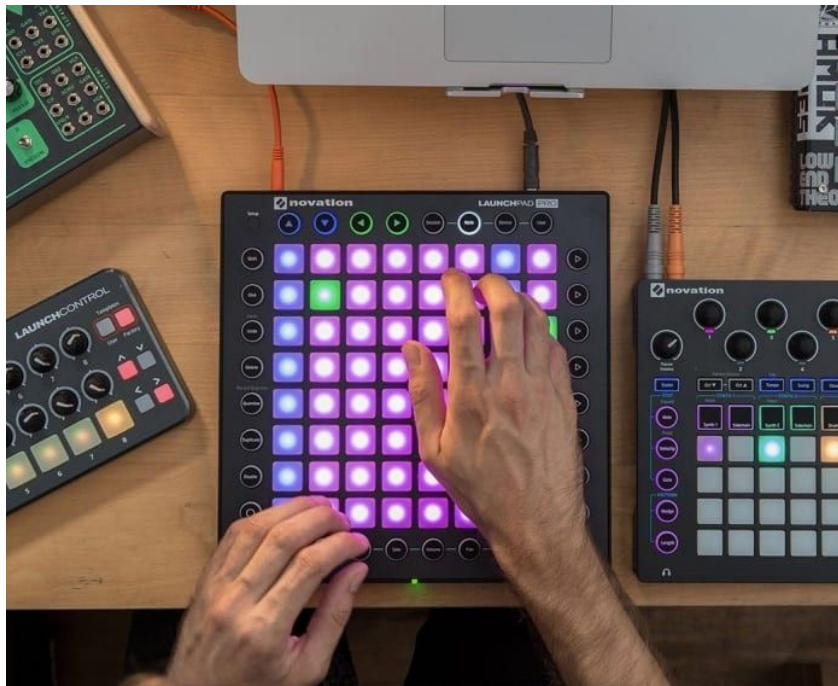
More this



Less this



Made for live play



Inspired by performance mode of DAWs
Audio reactivity
MIDI I/O
Grid MIDI controllers

Coming:
Open Sound Control (OSC)
Syphon/Spout support (probably...)

Photo from Novation

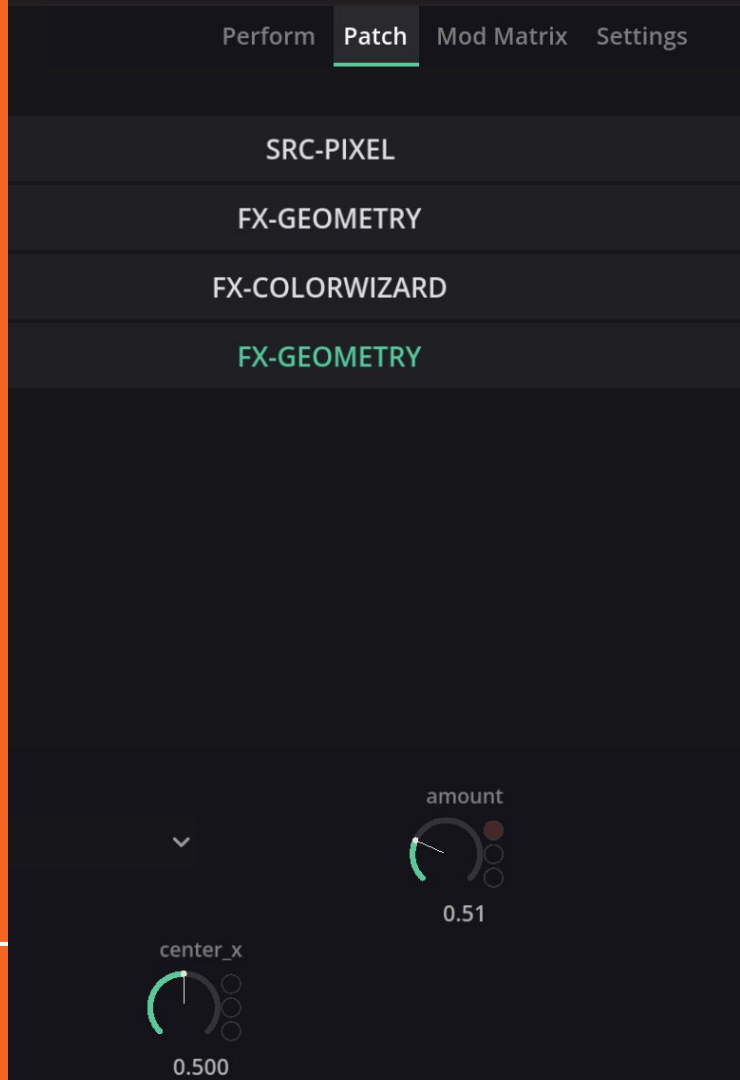
Modular at heart

bitMoire modules can be interconnected and customized in different ways to achieve unique visuals.

Instead of utilitarian tools, modules are self-contained "instruments" similar to modular synthesizers.

Goal: *Easy to learn, difficult to master*

Modules are very work in progress



Dev Stack

MacOS / Apple M2 Pro 16GB

Claude Code -> Pi Coding Agent

Coding LLM: z.AI GLM 5.1

Brainstorming: Gemini 3.5-Flash/Pro

Proof of concept

Instead of building the whole thing in one go, I needed to make a PoC first

>>> Goal: Create a simplified model of the entire application

- Hardcoded shader network
- Simple parameter control
- No modulation, tempo sync or other fancy features
- If it works - great; if not - at least I didn't waste days

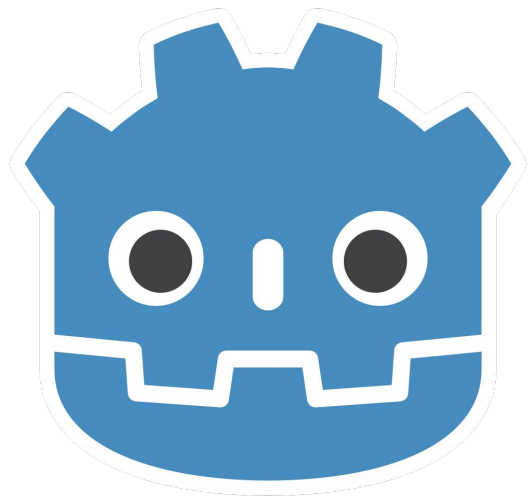
Tried openFrameworks (C++) - no luck with macOS Metal shaders
Game engines already offer multi platform support with GUI, localization, shaders and all. Had to pick between Godot, Unity or Unreal Engine.

>>> Godot prototype worked with GUI ✨



Phase 1

Godot + GDScript



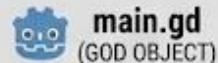
Initial implementation in GDScript

Available features:

- Core rendering with modules, patches, etc
- Performance mode
- Basic UI implementation with localization, editor pages, styling
- File load/save
- MIDI and pad controller integration
- Settings, Undo, Redo, Keyboard shortcuts
- Logging, debugging, tests

Problem: Any change could broke the application due to fragile architecture.
GDScript limitations became a bottleneck.

ENTRY & INITIALIZATION



_ready() initializes:

- All controllers
- Signal wiring
- Scene tree ownership

Creates & Wires
(arrow to Column 2)

_process() drives loop,
_notification() handles
resize and termination

GLOBAL SERVICES & NAVIGATION

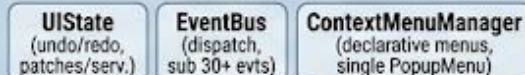
AUTOLOAD SINGLETONS



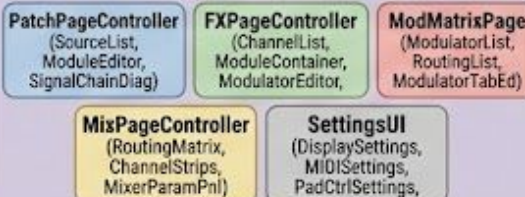
NAVIGATION LAYER



CORE STATE

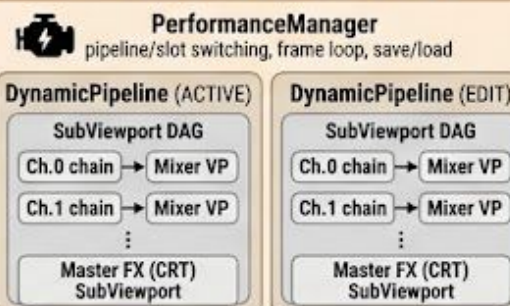


PAGE CONTROLLERS

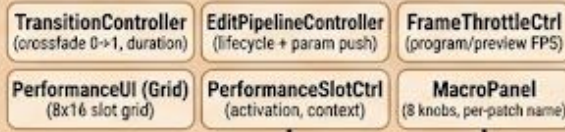


CORE ENGINE & EXECUTION

PERFORMANCE ENGINE LAYER



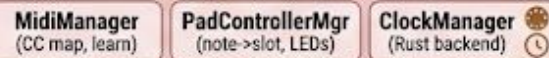
PIPELINE CONTROLLERS & UI



PARAMETER DISPATCH

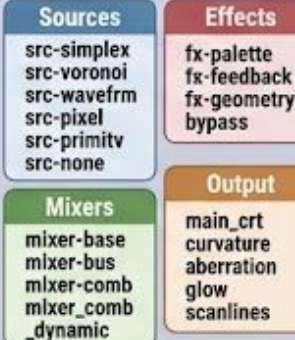


MIDI / CLOCK



RESOURCES & UTILITIES

SHADERS / SCENE TREE



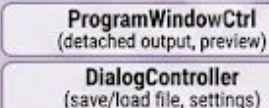
Transitions

crossfade/cut/dissolve/wipe_h/wipe_v

MONITORING / DEBUG

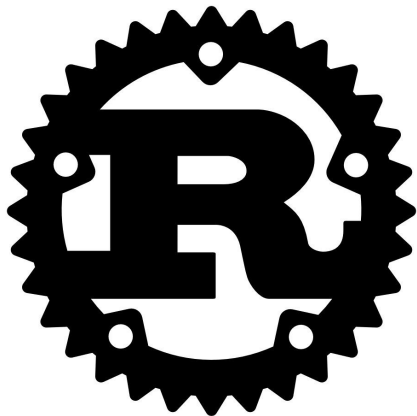


WINDOW / DIALOG



Phase 2

Godot + Rust



**The Rust
Programming
Language**

Full rewrite in Rust

Initially asked the agents to migrate **everything**

Agents lied that full migration is not possible

Found missing functions in **gdrust** docs

Migration took **weeks** at 10+ hours a day

Core engine wasn't working most of the time

Plan > Human Review > Implementation

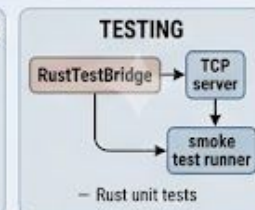
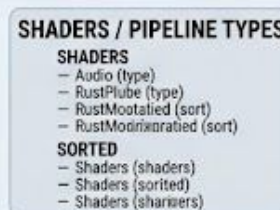
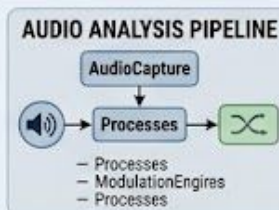
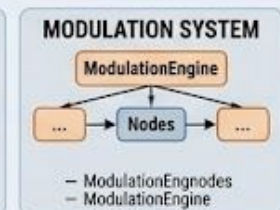
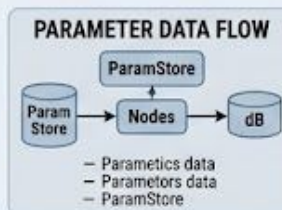
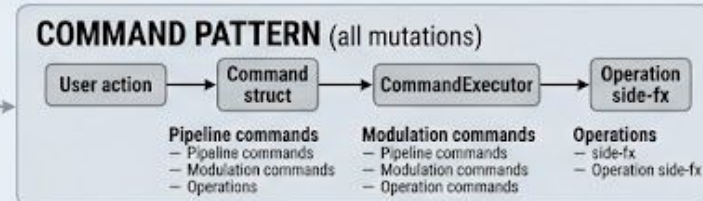
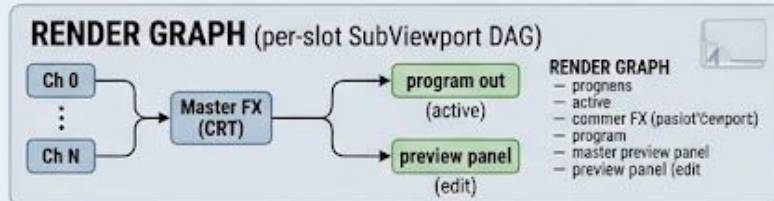
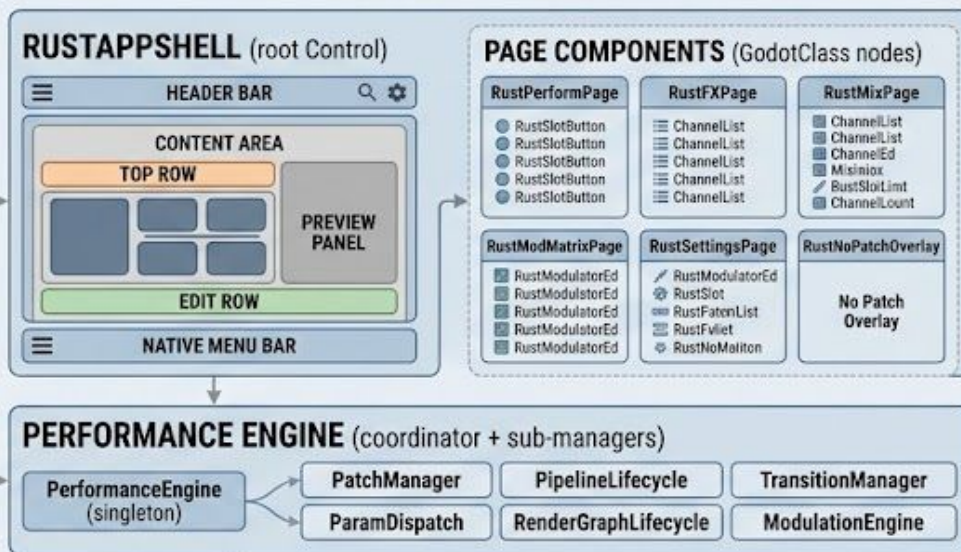
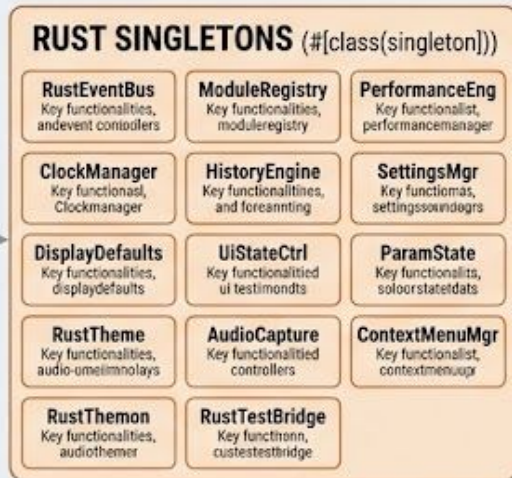
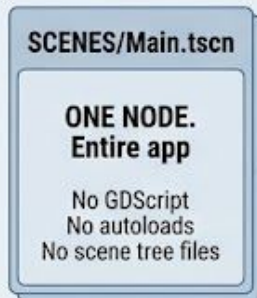
Lot of manual work testing changes

Core work finished a couple of days ago,

but still not 100% done 🦴

GODOT ENGINE (runtime only)

Architecture



	Pure GDScript	Current Rust	Delta
Application Code			
GDScript	31,866 lines	0	-31,866
Rust	0	76,657 lines	+76,657
Total app code	31,866	76,657	+44,791
Test Code			
GUT (GDScript)	25,658 lines (116 files)	0	-25,658
Rust <code>#[test]</code>	0	3,905 lines	+3,905
Python smoke	0	16,751 lines (53 files)	+16,751
Total test code	25,658	20,656	-5,002
Shaders	791 lines (21 files)	2,247 lines (28 files)	+1,456
GodotClass types	0	56	+56

	Pre-Rewrite (GDScript)	Current (Rust)	Delta
App code	61,012 lines	76,657 lines	+15,645
GDScript	27,821	0	-27,821
Rust	33,191	76,657	+43,466
Test code	31,666	20,656	-11,010
GUT (GDScript)	21,849	0	-21,849
Rust tests	(embedded)	3,905	+3,905
Python smoke	9,817	16,751	+6,934
Shaders	1,519	2,247	+728
Files			
GDScript	90	0	-90
Rust <code>.rs</code>	97	173	+76
GodotClass types	21	56	+35

Test harness

Tests running on code

- **GUT** - Godot unit tests (old)
- **Rust Compiler**
- **Clippy** - Rust linter
- **Python** for custom violations
(hardcoded text, style,
non-localized strings, etc.)

UI tests checking application

- Custom **Python** smoke tests
 - TCP connection to app
 - Imitates user actions
 - Calls Godot functions
 - Evaluates responses from app

Saves me days of manual work and catches regressions introduced by new features.

Problems during development

Stutters due to sync/async calls
Dual active/edit pipeline confusion
Flaky tests and wrong assumptions
Type safety violations
Regression bugs
OS level quirks (APIs, permissions)

Patch 1 Perform Patch Mod Matrix Settings Pause Stop 120.0 89.1.1 00:02:58

CHANNELS + CHAIN

BUS 1 SRC-PIXEL ×

▶ MAIN FX-GEOMETRY ×

FX-GEOMETRY ×

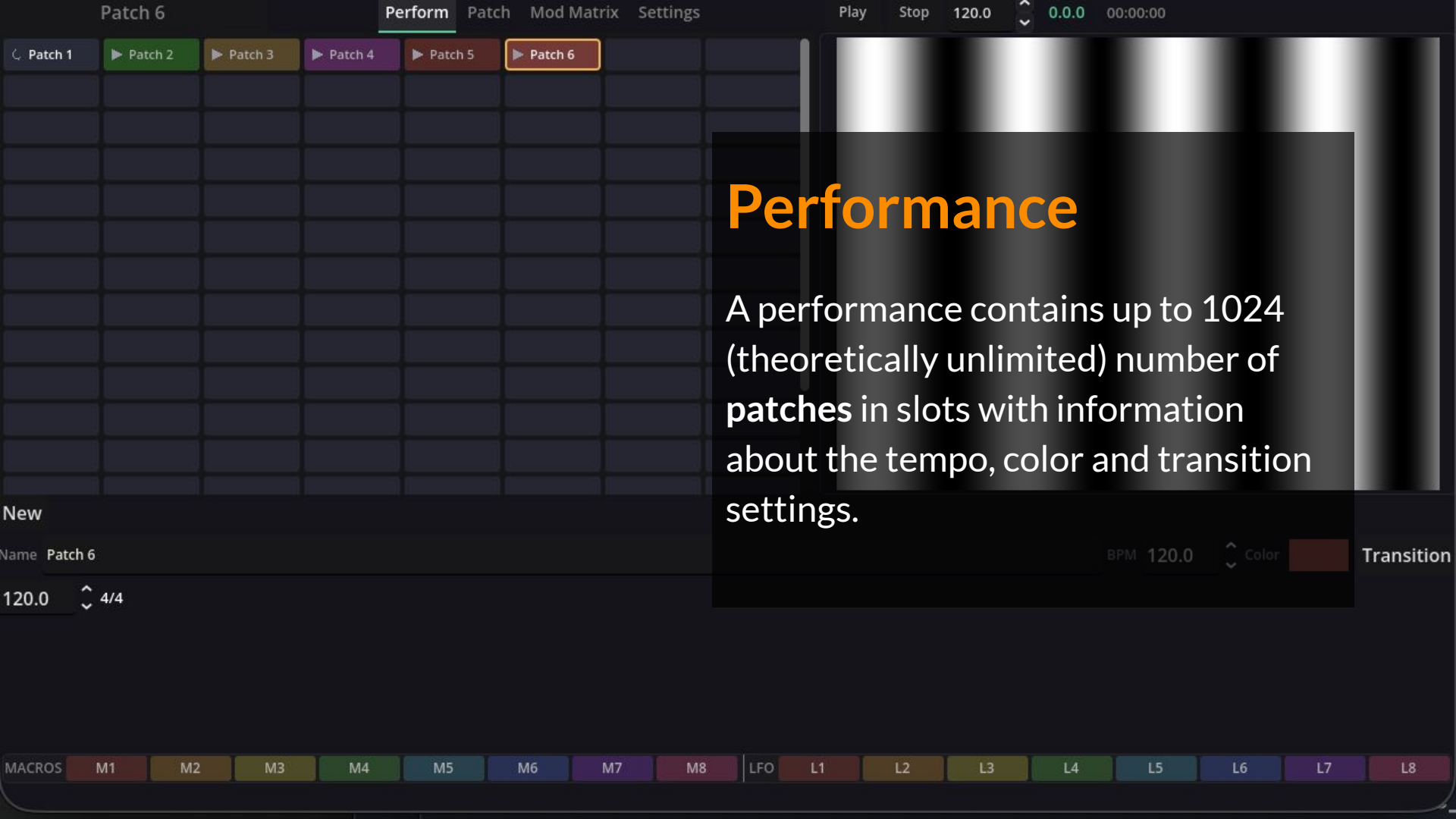
Tunnel sub_type speed 1.68

palette_shift 0.280 quantize 0.000 tunnel_twist 3.0

MACROS M1 M2 M3 M4 M5 M6 M7 M8 LFO L1 L2 L3 L4 L5 L6 L7 L8

Patch

- **Modules** generate or modify video.
- **Channels** contain modules, can mix and route to other channels.
- **Modulators** change parameters dynamically at each render step.
- **Routings** determine modulation source, target and depth



Performance

A performance contains up to 1024 (theoretically unlimited) number of **patches** in slots with information about the tempo, color and transition settings.

Let's try **bitMoire!**

DEMO

Questions?

Thank you!

@cursortoxyz
